MNIST Handwritten Digit Classification

A comparison of Single-Layer, Multi-Layer, and CNN (LeNet-style) Models

Learning Objectives

By the end of this assignment, students will be able to:

1. Understand the MNIST handwritten digit classification problem.
2. Be familiar with the rolling digit recognizer competition on Kaggle
3. Use Kaggle dataset and notebooks for implementing their first few classification models
4. Describe, Implement and compare:
    - A **single-layer neural network**
    - A **multi-layer perceptron (MLP)**
    - A **convolutional neural network (CNN)** based on the **LeNet architecture**
5. Analyze the effect of depth, non-linearity, and convolution on classification performance.
6. Interpret training curves, confusion matrices, and learned features.
7. Submit their first few entries on Kaggle
8. Produce a report using LaTex on overleaf

Background

The MNIST dataset consists of grayscale images of handwritten digits (0–9), each of size 28×28 pixels. The task is to classify each image into one of ten digit classes. This assignment explores how increasingly powerful neural architectures improve classification accuracy and generalization by participating in the digit recognition competition on Kaggle.

Assignment Details

A. Single-Layer Neural Network (Perceptron) (30 Marks)

1. Create a detailed description of what classification means using real world examples. Add images and figures to describe the classification problems in various domains including Compute Vision, Language Processing, and Medical etc.
2. Learn and describe, why writing programs to automatically classify objects proved to be a tough problem. Take hand-written character recognition as the primary case for this description.
3. Introduce the Perceptron model for classification (no biological inspiration please) by describing it's working. Describe it as a linear model for classification and describe various interpretations of the weights like a template, a vector normal to the separating hyper-plane. Furthermore inspire the bias term as a negative of threshold or as an offset of separating hyper-plane from the origin.
4. Using perceptron as the basic model, introduce the data driven or supervised learning paradigm for creating good classifiers. Properly introduce the loss function(s) and how do we put the learning of optimal parameters as optimization problem.

5. Discuss the representational limitations of a single-layer model and some possible ways to address these limitations. Describe ways of extending the basic perceptron to handle multi-category classification problems.
6. Describe the MNIST Digit dataset for creating image classifiers giving some description of why MNIST is a good benchmark dataset for image classification and what challenges handwritten digit recognition pose compared to typed characters?
7. Present details of a **PyTorch or TensorFlow/Keras** based implementation of Perceptron for classifying MNIST digits using softmax activation and cross-entropy loss.
8. Present a detailed description of results including training error & loss function curves, test accuracy etc.

B. Multi-Layer Perceptron (MLP) (25 Marks)

1. Extend the basic perceptron model into a multi-layer perceptron model clearly presenting the working of each layer as product of a matrix and an input vector hence describing the overall model working as algebraic products. Try to give an intuitive explanation of how adding hidden layers might increases model capacity.
2. Describe the role of activation functions in it starting with linear activation and compare sigmoid, tanh, and ReLU activation functions.
3. Explain the role of loss function and how back-propagation help in finding good parameters of an MLP.
4. Present details of a **PyTorch or TensorFlow/Keras** based implementation of the MLP for classifying MNIST digits justifying the number of hidden layers, learning rate and optimizer used.
5. Present a detailed description of results including training error & loss function curves, test accuracy etc.

C. Convolutional Neural Networks (25 Points)

1. Introduce CNN and draw and label the **LeNet-style CNN architecture** used by LeCun for character recognition.
2. Describe the purpose of each layer in LeNet:
    - Convolution
    - Pooling
    - Fully connected layers
3. Explain the key ideas behind CNNs:
    - Local receptive fields
    - Weight sharing
    - Feature maps
4. Why are CNNs better suited for image data than MLPs? Justify by describing the spatial structure in images and how CNN make use of it in the convolution layers.
5. Present details of a **PyTorch or TensorFlow/Keras** based implementation of the CNN for classifying MNIST digits. Use the standard LeNet architecture in your implementation.
6. Present a detailed description of results including training error & loss function curves, test accuracy etc.
7. Finally present a detailed performance analysis and comparison of the three classifier you trained in the first three parts

D. Entries of Kaggle (20 Points)

Use your Kaggle account to train and submit your entries (minimum 1 for each classifier) for the MNIST Digit challenge available at **https://www.kaggle.com/competitions/digit-recognizer.** You must use kaggle notebook and platform to train your model and share your implementation and kaggle entry details.